

Launch Features (MVP)

Core Music Library Strong

A comprehensive local music library that supports importing, organizing, and playing high-resolution audio files with full metadata management capabilities.

- Import music from Files app or external drives
- Scan, index, and parse metadata (ID3, Vorbis, .cue support)
- Organize library with sorted views (Artists, Albums, Genres, File Quality)
- Support for FLAC, ALAC, AIFF, WAV, APE, DSD formats
- Embedded tag parsing and editing

Tech Involved

- SwiftUI 5 for UI components
- MVVM architecture with ObservableObject pattern
- SwiftData with Core Data fallback for metadata indexing
- SFBAudioEngine and FFmpegKit for extended format support
- TagLib for metadata parsing

Main Requirements

- Fully functional offline operation
- Sandboxed local processing and storage
- Support for libraries up to 10,000 tracks
- Compliance with Apple HIG 2025
- WCAG 2.2 AA accessibility compliance

Advanced Playback Engine Strong

A high-fidelity audio playback system capable of bit-perfect reproduction with extensive format support and audio enhancement capabilities.

- Lossless, gapless playback with waveform visualization
- High-resolution format badges and quality indicators
- 10-band equalizer with presets
- Gain control and replay gain support
- Crossfade and audio output settings

Tech Involved

- AVFoundation for primary playback
- SFBAudioEngine for DSD and APE support
- FFmpegKit for additional codec support
- Swift async/await for concurrent processing
- Custom DSP pipeline for audio effects

Main Requirements

- Bit-perfect output when supported by hardware
- Sample rate conversion for DAC output
- Minimal CPU/battery usage during playback
- Background audio support with iOS audio session management
- Waveform rendering with resolution indicators

Smart Playlist & Filtering Strong

A powerful system for creating dynamic playlists based on metadata attributes, allowing users to organize their music collection with precision.

- Create smart playlists with multiple filter criteria
- Filter by audio quality, genre, artist, and custom tags
- Save and manage multiple playlist configurations
- Sort and group results by various attributes
- Quick filtering within current view context

Tech Involved

- SwiftData query predicates
- Combine framework for reactive updates
- Custom filter rule engine
- Persistent playlist storage

Main Requirements

- Support for complex boolean logic (AND, OR, NOT)
- Real-time filtering updates
- Persistent playlist configurations
- Performance optimization for large libraries
- Intuitive filter creation interface

Metadata Editor Strong

A comprehensive tool for viewing and editing audio file metadata, including tags, artwork, and lyrics, with batch editing capabilities.

- Edit ID3, Vorbis, and other tag formats
- Manage embedded lyrics and album art
- Support for cue sheets and track splitting
- Batch editing for multiple files
- Custom tag field support

Tech Involved

- TagLib integration
- FFmpeg metadata tools
- Custom metadata parser/writer
- Image processing for artwork

Main Requirements

- Non-destructive editing with original backup
- Support for all major tag formats
- Embedded lyrics display during playback
- High-resolution artwork management
- Batch operations for efficiency

Future Features (Post-MVP)

Cloud Integration

- Connect to Google Drive, Dropbox, OneDrive via OAuth
- Stream music directly from cloud storage
- Sync library metadata across devices
- Encrypted local cache for offline playback
- Background downloading for offline availability

Tech Involved

- OAuth 2.0 authentication
- FileProvider framework integration
- Custom streaming buffer management
- Background transfer service
- Encrypted local storage

Main Requirements

- Seamless switching between local and cloud libraries
- Bandwidth-aware streaming quality adjustment
- Efficient caching strategy
- Conflict resolution for metadata changes
- Privacy-focused authentication flow

Streaming Service Connectivity

- Integration with high-quality streaming services (e.g., Qobuz)
- Unified library view combining local and streaming content
- Smart download management for offline listening
- Playlist synchronization with streaming services
- Discovery features based on local library

Tech Involved

- Streaming service APIs
- Adaptive bitrate streaming
- OAuth authentication
- Custom metadata reconciliation
- Background fetch for library updates

Main Requirements

- Unified search across local and streaming content
- Quality-aware playback decisions
- Bandwidth monitoring and adaptation
- User account management (optional)
- Playlist import/export capabilities

Advanced Audio Processing

- Room correction and acoustic compensation
- Headphone-specific EQ profiles
- Spatial audio and 3D sound processing
- Advanced DSP effects (compression, limiting, etc.)
- Audio visualization options

Tech Involved

- Core Audio processing

- Custom DSP algorithms
- Spatial audio frameworks
- Metal for visualization rendering
- Audio Unit extensions

Main Requirements

- Device-specific audio profiles
- Real-time processing with minimal latency
- User-adjustable parameters
- Preset management system
- Hardware acceleration where available

Social & Sharing Features

- Local network library sharing
- Playlist export and sharing
- Last.fm scrobbling (optional)
- Listening statistics and reports
- Music recommendation engine (on-device)

Tech Involved

- Local network discovery (Bonjour)
- Export formats (JSON, XML, M3U)
- Last.fm API integration
- Core ML for on-device recommendations
- SwiftData for statistics tracking

Main Requirements

- Privacy-focused sharing (no account required)
- Opt-in for any external services
- Anonymized data collection (if enabled)
- Local-only recommendation system
- Exportable listening history

System Diagram

[This would be an SVG architecture diagram with color-coded layers, rounded containers, and clear component relationships]

The architecture follows a modular design with these key components:

1. Presentation Layer

2. SwiftUI Views (Library, Player, Settings)

3. ViewModels (MVVM pattern)

4. UI State Management

5. Accessibility Features

6. Domain Layer

7. Music Library Manager

8. Playback Controller

9. Metadata Service

10. Smart Playlist Engine

11. Audio Processing Pipeline

12. Data Layer

13. Local Storage (SwiftData/Core Data)

14. File System Manager

15. Cloud Storage Adapters

16. Streaming Service Adapters

17. Caching System

18. Core Services

19. Audio Engine (AVFoundation + Extensions)

20. Format Decoders (SFBAudioEngine, FFmpegKit)

21. Metadata Parser/Writer

22. DSP Processing Chain

23. Background Task Manager

24. External Integrations (Post-MVP)

25. Cloud Storage Providers

26. Streaming Services

27. Local Network Services

28. Optional Analytics

Questions & Clarifications

- What specific hardware compatibility issues have you encountered with high-resolution audio formats on iOS?
- Are there any specific audio processing features (beyond EQ and gain) that are critical for your target audience?
- How important is battery optimization compared to audio quality? Are there scenarios where users would prioritize one over the other?
- What level of customization do you want to provide for the UI? Should users be able to rearrange library views or customize the player interface?
- Do you have any specific requirements for handling very large album artwork or embedded PDF booklets?
- How should the app handle situations where metadata is inconsistent or missing?
- What are your thoughts on implementing AirPlay 2 and CarPlay support in the MVP versus post-MVP?

List of Architecture Consideration Questions

- How should we balance local storage usage versus processing time for waveform generation and audio analysis?
- What's the optimal database schema design to support efficient filtering and smart playlists while maintaining performance with large libraries?
- Should we implement a plugin architecture for audio format support to keep the core app size smaller?
- How can we design the cloud integration to maintain the offline-first philosophy while still providing seamless access to cloud content?
- What's the best approach for handling audio format conversion when necessary (e.g., for hardware compatibility)?
- How should we architect the audio engine to support both local playback and streaming with minimal code duplication?
- What caching strategies would be most effective for balancing performance and storage usage?
- How can we design the metadata system to be extensible for future format support while maintaining backward compatibility?
- What's the optimal threading model for audio processing to ensure smooth UI performance during intensive operations?
- How should we approach error handling and recovery, particularly for file access and cloud connectivity issues?